

Resumen global: Tareas 10 al 15

Resumen tarea 10

1. El Problema del Prototipado y la Solución de Gradio

La creación de aplicaciones con **frameworks modernos** requiere una cantidad considerable de código. **Gradio** permite construir **interfaces completas** con una sola línea de código en Python, priorizando la simplicidad y velocidad.

2. Cimientos Técnicos y Seguridad

Para utilizar Gradio debemos de realizar unos pasos previos:

1. Escribir el import `import gradio as gr`.
2. Creas un archivo llamado `.env` donde **escribirás las claves** de las APIs
3. Para **cargar** claves desde archivos `.env` se utiliza `load_dotenv(override=True)`

3. Flujo de Trabajo: De la Lógica a la Interfaz

La interfaz requiere de tres elementos clave:

1. **Motor (`fn`)**: La lógica de backend que se ejecuta
2. **Entrada (`inputs`)**: Define cómo se **reciben los datos**
3. **Salida (`outputs`)**: Define dónde y cómo se **muestra el resultado**

4. Arquitectura y Ejecución

Gradio opera bajo una arquitectura que equilibra la **accesibilidad pública** con la **seguridad local**.

Se ejecuta mediante la línea `.launch(share=True)` que genera un **enlace público temporal** (72 horas) que tuneliza la conexión hacia el servidor local.

El modo `flagging_mode='never'` permite **desactivar** el botón 'Flag', evitando que se guarden datos de usuarios en carpetas locales (CSV).

Gradio solo funciona como el transmisor del texto entre el usuario y el servidor local.

5. Casos de Uso: Del Procesamiento Simple a la Integración con GPT

Una lógica simple de backend donde una entrada de texto en **minúsculas se transforma en mayúsculas** mediante `.upper()`.

- **Input:** 'hola' → **Motor:** `shout()` → **Output:** 'HOLA'.

Integración Avanzada con OpenAI

Se puede cambiar el motor de la aplicación sin alterar la estructura de la interfaz de Gradio.

- La función `message_gpt(prompt)` utiliza el modelo `gpt-4o-mini`.
- Define roles de `'system'` y `'user'`.
- Se pueden ajustar parámetros visuales como etiquetas y tamaño de los cuadros de texto:
 - `inputs=[gr.Textbox(label='Tu Mensaje:', lines=6)]`
 - `outputs=[gr.Textbox(label='Respuesta:', lines=8)]`

Este nivel de integración permite que la respuesta sea generada por la API de OpenAI y sea mostrada por Gradio.

Resumen tarea 12

1. El Problema: El "Cerebro Aislado"

Los modelos LLM actuales **no pueden acceder** a bases de datos privadas, es decir, no conocen datos en tiempo real y no pueden ejecutar código por si mismos.

2. La Solución: Function Calling

La tecnología de **Function Calling** dota a la IA de la capacidad de decidir cuándo es necesario **solicitar** la ejecución de una herramienta externa.

3. Arquitectura de FlightAI

La estructura del asistente se basa en un flujo de tres componentes interconectados: **Definir** → **Describir** → **Ejecutar**.

Componente	Función Principal
Python	Contiene el código que consulta la base de datos de precios.
Schema JSON	Proporciona al LLM una descripción técnica de qué hace la herramienta y qué parámetros requiere.
Handler	Gestiona la comunicación, decodifica la solicitud de la IA y devuelve el resultado.

4. Proceso de Implementación Técnica

Paso 1: Materiales (Datos y Lógica)

Se establece una base de datos ficticia (`ticket_prices`) con destinos y precios. Se implementa la función `get_ticket_price` que obtiene el precio de los aviones manejando mayúsculas/minúsculas mediante `.lower()` .

Paso 2: El Manual de Instrucciones (Schema y Registro)

El LLM requiere un esquema JSON (`price_function`) que defina el **nombre** y una **descripción** de la función, en los **parámetros** se definen el **tipo** del dato requerido y finalmente la función se añade a una **lista de tools** que se envía a la API de OpenAI.

Paso 3: El Puente (Decodificación y Respuesta)

Cuando la IA requiere a la herramienta, envía un **mensaje tipo String** que debe ser transformado con `json.loads()` para convertirlo en un diccionario.

Se llama a la función de búsqueda `get_ticket_price` y finalmente se crea un objeto de respuesta con el rol `tool` , el contenido en formato JSON y un `tool_call_id` .

Paso 4: Ciclo de Vida de la Petición

El proceso sigue un orden lógico circular:

1. **Entrada del Usuario:** "¿Precio para Berlín?"
2. **Decisión del LLM:** Identifica la necesidad de usar la herramienta y emite un *Stop & Request*.
3. **Acción del Handler:** Ejecuta el código Python y obtiene el resultado (\$499).

4. **Generación de Respuesta:** El LLM recibe el dato y genera el mensaje final para el usuario: "El billete cuesta \$499".

5. Protocolo de Inspección: Pruebas Obligatorias

Para validar la integridad del asistente, el sistema debe superar tres escenarios críticos:

1. **Éxito:** Confirmar que el JSON se **lee correctamente** y que los precios se recuperan con **precisión** del diccionario.
2. **Dato no disponible:** Si el destino es inexistente la función interna debe devolver `Unknown` sin inventarse ningún valor.
3. **System Prompt:** Validar que el `system_message` está activo.

6. Guía de Resolución de Problemas (Debugging)

- **Errores JSON:** Revisar **comas** y **llaves** en el esquema de la función.
- **Alucinaciones:** Si la IA inventa datos, es necesario **mejorar la descripción** en el esquema JSON para que sea más clara sobre los límites de la herramienta.
- **Tipos de Datos:** El campo `content` en el `handle_tool_call` debe devolver siempre un *String*.

Resumen tarea 13

1. El Problema de la Herramienta Única

El sistema asume que la entrada del usuario era una **solicitud de precio** utilizando la función `processQuery` que verifica si la consulta incluía la palabra "price". Pero ante **otro tipo de consultas** el sistema fallaba.

2. Arquitectura del Sistema Multi-Agente

Para resolver las limitaciones se ha implementado una arquitectura que cambia la forma en que el asistente "piensa" y procesa la información.

Componente	Función
Input Usuario	Escribe la consulta en lenguaje natural.

Cerebro (LLM)	Analiza la pregunta del usuario utilizando un modelo de lenguaje.
Router	Decide qué herramienta activar basándose en el análisis de intención.
Ejecución	Llama a la función específica: <code>get_ticket_price</code> o <code>get_flight_status</code> .
Respuesta Final	Sintetiza el resultado de la herramienta para el usuario.

3. Proceso de Implementación Técnica

La transformación del asistente se llevó a cabo a través de cinco pasos técnicos fundamentales:

Paso 1: La Capa de Datos

Se creó una base de datos simulada (`flight_status_db`) utilizando una estructura de clave-valor.

Paso 2: Lógica de Recuperación y Seguridad

Se creó la función `get_flight_status` como puente entre la IA y los datos.

El sistema no debe fallar si el vuelo no existe; para ello, se utiliza el método `.get()` que devuelve `Status Unknown` en caso de claves inexistentes.

Paso 3: Esquema JSON

Se definió un esquema JSON para explicar lo que hace la función `get_flight_status`

Paso 4: Registro de Herramientas

Se actualizó la herramienta `tools`.

Paso 5: Lógica de Enrutamiento (El Cerebro)

- **IF:** Si la IA pide `get_ticket_price` → Ejecutar lógica de precios.
- **ELIF:** Si la IA pide `get_flight_status` → Ejecutar nueva lógica de estado.

4. Validación del Sistema

Pruebas realizadas:

1. Se verifica que las capacidades originales no empeoraran.

2. Se comprueba que la nueva **función ejecuta correctamente**.

5. Capacidades Avanzadas: Consultas Híbridas

El modelo es capaz de detectar **dos intenciones** distintas en una sola frase y **ejecutar múltiples llamadas secuenciales**.

Resumen tarea 14

Infraestructura y Arquitectura Tecnológica

La productora autónoma se fundamenta en un ecosistema de herramientas integradas, cada una desempeñando un rol crítico en el flujo de producción:

Componente	Herramienta	Función Principal
El Cerebro	TinyLlama-1.1B-Chat	Analiza el texto original y extrae las acciones principales en inglés.
El Pintor	Realistic Vision V5.1	Modelo especializado en fotografía y rostros realistas .
El Locutor	Edge-TTS	Genera la narración utilizando voces neuronales de Azure.
El Traductor	Deep-Translator	Traduce el análisis del cerebro (inglés) a la narración final (español).
El Editor	MoviePy	Realiza el montaje , sincronización de audio/imagen y renderizado final.

Configuración del Entorno (*Environment Setup*)

Para garantizar la eficiencia operativa, el sistema implementa una lógica de detección de hardware:

- **Prioridad de GPU:** El script detecta si **CUDA** está disponible para acelerar el procesamiento.
- **Optimización de Memoria:** El uso de `torch.float16` para **ahorrar VRAM en la GPU**.
- **Gestión de Recursos:** En casos de errores de memoria, se recomienda ejecutar `flush()` y `gc.collect()` entre escenas.

El Ciclo de Producción: Fases del Pipeline

Fase 1: Ingestión y Segmentación (El Guionista)

Se utiliza una estrategia de **Chunking** donde se divide el el texto en escenas.

Fase 2: Análisis y Dirección de Arte (El Cerebro)

Se utiliza **Prompt Engineering** para transformar el texto crudo en instrucciones visuales

Fase 3: Generación Visual y de Audio (El Artista y El Locutor)

1. **Generación Visual:** Se utilizan "listas negras" para evitar errores estructurales, se asigna una temperatura de 0.7 y se detalla un poco la descripción de los planos.
2. **Generación de Audio:** El texto se traduce al español con Google Translator, se utiliza **Edge-TTS** para las voces y la generación de audio se realiza mediante funciones asíncronas (`async/await`).

Fase 4: Montaje y Orquestación (La Sala de Montaje)

Se ajusta automáticamente la **duración de la imagen** para que coincida con la duración del audio generado (`set_duration`). Se aplica `crossfadein(0.5)` para suavizar los cortes entre escenas.

Solución de Problemas y Nivel Experto

El documento identifica **fallos comunes** y sus **soluciones técnicas**:

- **Caras deformes:** Aumentar `num_inference_steps` o asegurar el uso de `close-up` en el prompt.
- **Inconsistencia de personaje:** Revisar la función de `inyectar_vestuario` .
- **Audio desincronizado:** Verificar la **duración del clip** de imagen mediante `audio.duration` .

Resumen tarea 15

1. El Ecosistema de Hugging Face y Código Abierto

Hugging Face es una página donde encontramos modelos de IA generativa de forma *Open Source*. Sus componentes principales se dividen en tres categorías:

2. Configuración de Infraestructura y Seguridad

El despliegue exitoso de LLMs requiere una **preparación rigurosa** del entorno de ejecución, específicamente en plataformas como Google Colab.

2.1. Preparación del Hardware

Es obligatorio activar una GPU (T4) para **garantizar la velocidad y estabilidad del sistema**.

2.2. Gestión Segura de Credenciales

La seguridad se gestiona mediante el uso de **Tokens de Acceso** para evitar la exposición de contraseñas en el código visible, se utiliza la función "Secretos" de Google Colab:

- Se **crea un secreto** denominado `HF_TOKEN`.
- Se **accede** a él mediante la **librería** `google.colab.userdata`.
- Se activa el "Acceso al cuaderno" para permitir que la **máquina virtual utilice la llave** sin mostrarla.

3. El Proyecto FLUX.1: Generación de Imágenes

FLUX.1-schnell, es un modelo de **generación de imágenes a partir de texto**.

3.1. El Reto de la Memoria RAM

FLUX.1-schnell tiene un **tamaño > 30 GB**, mientras que la GPU T4 ofrece **15 GB**, por ello se utilizan **técnicas de optimización** para evitar el colapso del sistema.

3.2. Estrategias de Optimización ("Magia de Optimización")

Para solventar la limitación de hardware, se aplican tres técnicas críticas:

Técnica	Descripción Funcional
Compresión (bfloat16)	Uso de números decimales más pequeños (<code>torch.bfloat16</code>) para reducir el peso del modelo en memoria.
Sequential CPU Offload	Mueve partes del modelo entre la CPU y la GPU cargando solo lo necesario.
Slicing y Tiling	Corta la imagen en fragmentos procesables y habilita el procesamiento por partes

3.3. Ejecución y Reproducibilidad

- **Velocidad:** La versión *Schnell* solo requiere 4 "pasadas" generar el dibujo final.
- **Semilla (Seed):** El uso de `manual_seed(0)` asegura que el modelo genere exactamente la misma imagen ante el mismo texto.

4. Capacidades Avanzadas mediante Pipelines

Los **Pipelines** representan la API de alto nivel de Hugging Face, permitiendo **ejecutar inferencias complejas** con un código mínimo. Para un rendimiento óptimo es fundamental añadir el parámetro `device="cuda"`.

4.1. Análisis y Reconocimiento de Texto

Se evalúa el **tono emocional**, se identifican **nombres, organizaciones y ubicaciones** utilizando `aggregation_strategy="simple"` para evitar que la IA fragmente palabras en "tokens" inconexos.

4.2. Conocimiento y Traducción

Se extrae **respuestas exactas** de un contexto específico funcionando como un buscador inteligente.

Conversión de idiomas mediante **modelos especializados**.

4.3. Procesamiento de Audio (Text-to-Speech)

La **generación de voz** se produce mediante una **matriz matemática (onda)**.